UVM VERIFICATION OF A SYNCHRONOUS FIFO

SystemVerilog UVM Testbench, SVA Assertions & Functional Coverage

Prepared by

Salah Adel

Tools: QuestaSim • UVM • SystemVerilog Assertions

Digital Verification using SystemVerilog and UVM

Table of Contents

1. Project Overview 3
2. Verification Plan 3
3. UVM Testbench Architecture 4
4. How the Testbench Works 5
5. Data & Control Flow 5
6. Simulation Results 6
7. Coverage Reports 7
8. Assertions (Sequential-Domain Coverage) 8
9. Bug Report 9
10. QuestaSim Waveforms by Sequence 10
11. Summary 12

1. Project Overview

This project builds a complete UVM verification environment for a synchronous FIFO (16-bit data, depth 8) with write/read enables, a write acknowledge, and the status flags full, empty, almostfull, almostempty, overflow and underflow. The environment drives constrained-random stimulus through four sequences, predicts the expected behavior with a golden reference model, measures functional coverage, and checks the design with SVA assertions bound to the DUT.

Key Results

Item	Result
Functional coverage	100% — covergroups with wr_en x rd_en crosses against every status flag
Scoreboard	495 checks passed, 0 failed — UVM_ERROR = 0, UVM_FATAL = 0
Assertions	14 SVA properties — zero failures
Stimulus	Reset, 20 write-only, 20 read-only and 350 random write/read items (391 clock cycles)
Bugs found in the RTL	10
Simulator	QuestaSim

2. Verification Plan

Each design requirement is traced to a stimulus strategy, a functional-coverage item, and a check (golden model and/or assertion).

Label	Design Requirement	Stimulus Generation	Functional Coverage	Functionality Check
FIFO_1	Upon assertion of asynchronous reset (rst_n = 0), all internal pointers (wr_ptr, rd_ptr) and the counter (count) must reset, and the sequential outputs (data_out, wr_ack, overflow, underflow) must be cleared to 0.	Assert reset (rst_n = 0) at the beginning of simulation and randomly during runtime.	Covered in Reset Coverage	Golden Model & Assertion (a_reset_clear)
FIFO_2	Valid write operation: when wr_en = 1 and FIFO is not full, data is written into memory, wr_ptr increments, and wr_ack is asserted.	Randomization with WR_EN_ON_DIST weight (default 70%).	Cross coverage of wr_en x wr_ack	Golden Model & Assertion (a_wr_ack & a_wr_ptr_range)
FIFO_3	Valid read operation: when rd_en = 1 and FIFO is not empty, data is read from memory, rd_ptr increments, and data_out is updated.	Randomization with RD_EN_ON_DIST weight (default 30%).	Cross coverage of rd_en x data_out	Golden Model & Assertion (check_data in Scoreboard & a_rd_ptr_range)
FIFO_4	Full flag: when count == FIFO_DEPTH, the full flag must be asserted combinationally.	Directed & random write stimulus until memory fills completely.	Cross coverage of wr_en x rd_en x full	Golden Model & Assertion (a_full)
FIFO_5	Empty flag: when count == 0, the empty flag must be asserted combinationally.	Directed & random read stimulus until memory drains completely.	Cross coverage of wr_en x rd_en x empty	Golden Model & Assertion (a_empty)
FIFO_6	Almost-full flag: when count == FIFO_DEPTH - 1, the almostfull flag must be asserted.	Random/directed writes reaching depth - 1.	Cross coverage of wr_en x rd_en x almostfull	Golden Model & Assertion (a_afull)

Label	Design Requirement	Stimulus Generation	Functional Coverage	Functionality Check
FIFO_7	Almost-empty flag: when count == 1, the almostempty flag must be asserted.	Random/directed reads leaving only 1 entry.	Cross coverage of wr_en x rd_en x almostempty	Golden Model & Assertion (a_aempty)
FIFO_8	Overflow detection: attempting a write when FIFO is full (wr_en = 1 & full = 1) must trigger overflow without corrupting memory.	Random write requests generated while full = 1.	Cross coverage of wr_en x rd_en x overflow	Golden Model & Assertion (a_overflow_condition)
FIFO_9	Underflow detection: attempting a read when FIFO is empty (rd_en = 1 & empty = 1) must trigger underflow.	Random read requests generated while empty = 1.	Cross coverage of wr_en x rd_en x underflow	Golden Model & Assertion (a_empty_underflow)
FIFO_10	Simultaneous read & write: handling wr_en = 1 and rd_en = 1 together when empty (write wins) and when full (read wins).	Constraint-driven stimulus forcing wr_en = 1 & rd_en = 1 simultaneously.	Cross coverage of wr_en x rd_en combinations	Golden Model (Scoreboard) & Assertions (a_wr_ack, a_full_no_ack, a_empty_underflow)
FIFO_11	Pointer wrap-around & threshold: pointers and counter must wrap around to 0 after reaching FIFO_DEPTH - 1 and never exceed their limits.	Continuous sequences of writes and reads driving pointers through maximum depth.	Covered in Code and Covergroups	Assertions (a_wr_ptr_wrap, a_rd_ptr_wrap, a_count_range, a_wr_ptr_range, a_rd_ptr_range)

3. UVM Testbench Architecture

The top module instantiates the interface and the DUT and starts the UVM test. The environment contains the agent (sequencer, driver, monitor), the scoreboard, and the coverage collector, while the assertions module is attached to the DUT with a bind.

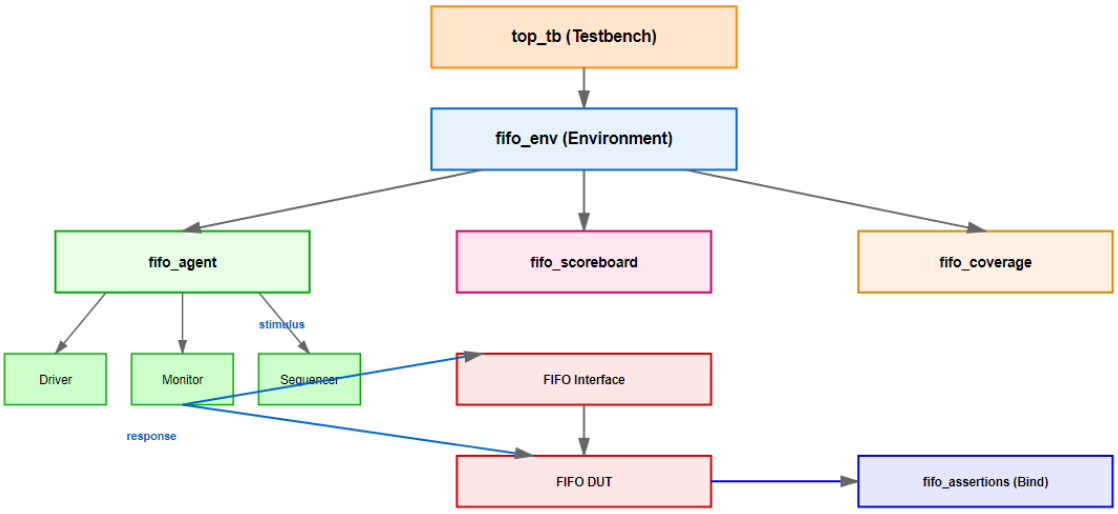


Figure 1 — UVM testbench structure.

4. How the Testbench Works

1. **Top module.** fifo_top generates a 10 ns clock, instantiates the FIFO_if interface and the FIFO DUT, binds the assertions module to the DUT (so it can also see the internal count, wr_ptr and rd_ptr), stores the virtual interface in the UVM configuration database, and calls run_test.
2. **Test and environment.** The test builds the environment, which builds the agent, the scoreboard, and the coverage collector, and then runs the four sequences one after another: reset, write-only, read-only, and simultaneous write/read.
3. **Driving the interface.** Each sequence creates sequence items with constrained-random rst_n, wr_en, rd_en and data_in. The driver takes an item from the sequencer, drives it onto the virtual interface, waits for the falling clock edge, and completes the item, so exactly one item is applied per clock cycle.
4. **Monitoring.** The monitor samples the inputs at the rising clock edge and the outputs (data_out, wr_ack, overflow, underflow, full, empty, almostfull, almostempty) at the following falling edge. This pairs every set of inputs with the DUT response they actually caused, and the monitor publishes the result through its analysis port.
5. **Analysis.** The scoreboard runs a golden reference model (a queue and a counter) on the sampled inputs and compares its prediction with the DUT outputs, including the data of every successful read. The coverage collector samples the same transactions and updates the covergroups. Independently of the UVM components, the bound assertions check the flags against the internal counter, the reset behavior, the acknowledge, overflow and underflow rules, and the pointers.

5. Data & Control Flow

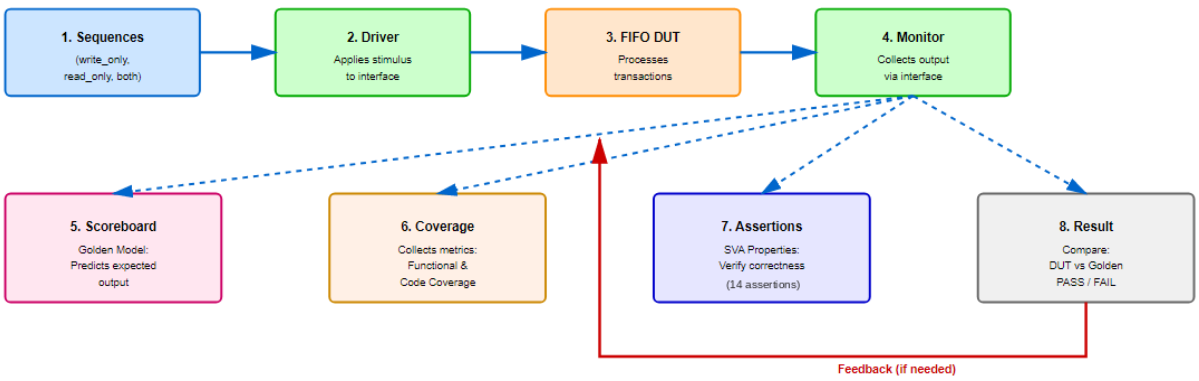


Figure 2 — Data and control flow from the sequences to the final PASS/FAIL result.

6. Simulation Results

The run ends at 3910 ns, when run_test calls $finish after all UVM phases have completed. The scoreboard reports 495 successful checks and 0 failed checks, and the UVM report summary shows 0 warnings, 0 errors and 0 fatals. The scoreboard performs one status-flag check for every transaction plus one data_out check for every successful read, which is why the count is higher than the number of clock cycles. The Covergroups window shows 100% coverage, and the sim window shows the UVM hierarchy: uvm_root, the test, the environment with the scoreboard, the coverage collector and the agent (sequencer, monitor, driver).

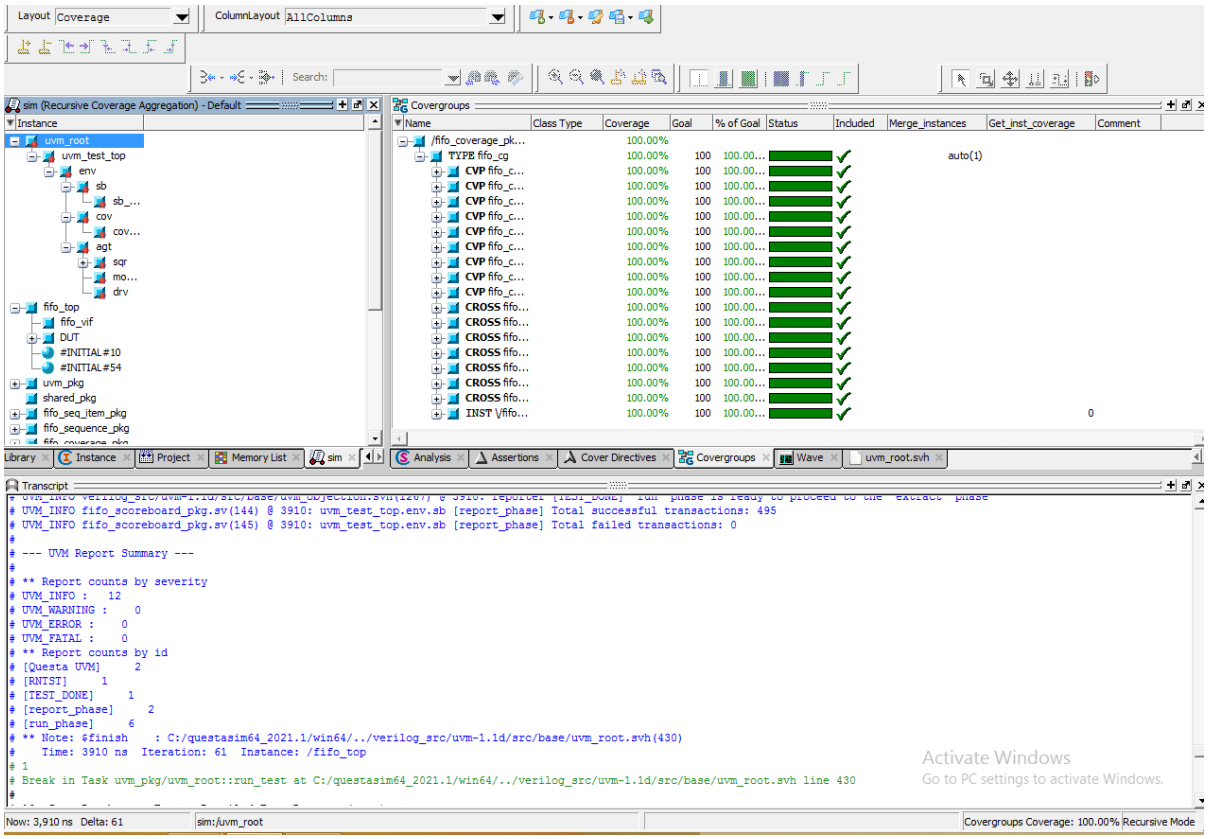


Figure 3 — End of simulation: UVM hierarchy, covergroups at 100%, scoreboard totals and UVM report summary.

7. Coverage Reports

7.1 Code Coverage

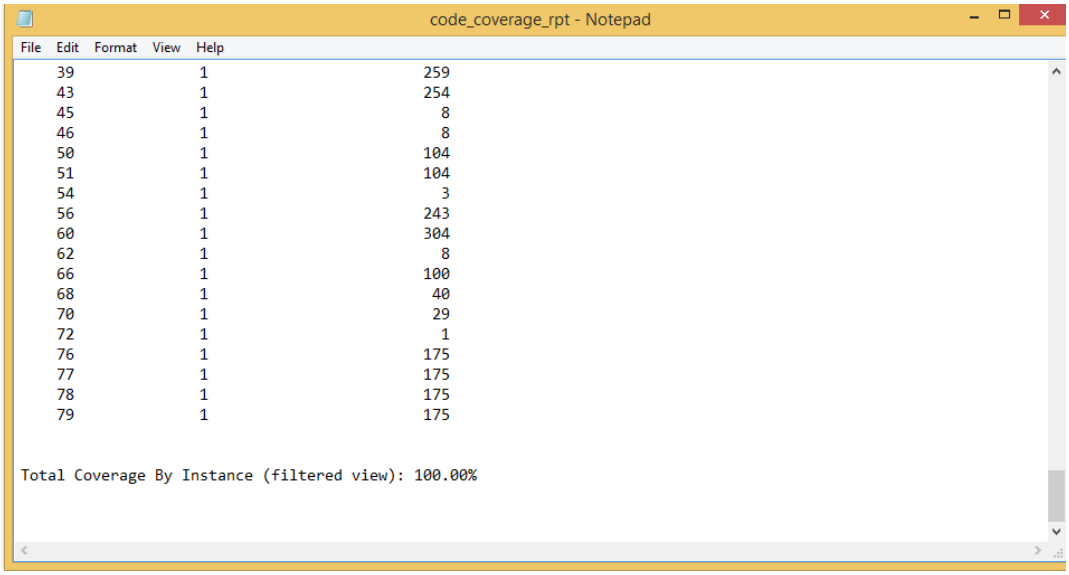


Figure 4 — Code coverage report.

7.2 Functional Coverage

The covergroup reports 100% total coverage. The crosses combine wr_en and rd_en with each status flag; combinations that cannot occur in the design are excluded with ignore bins, which show a hit count of zero.

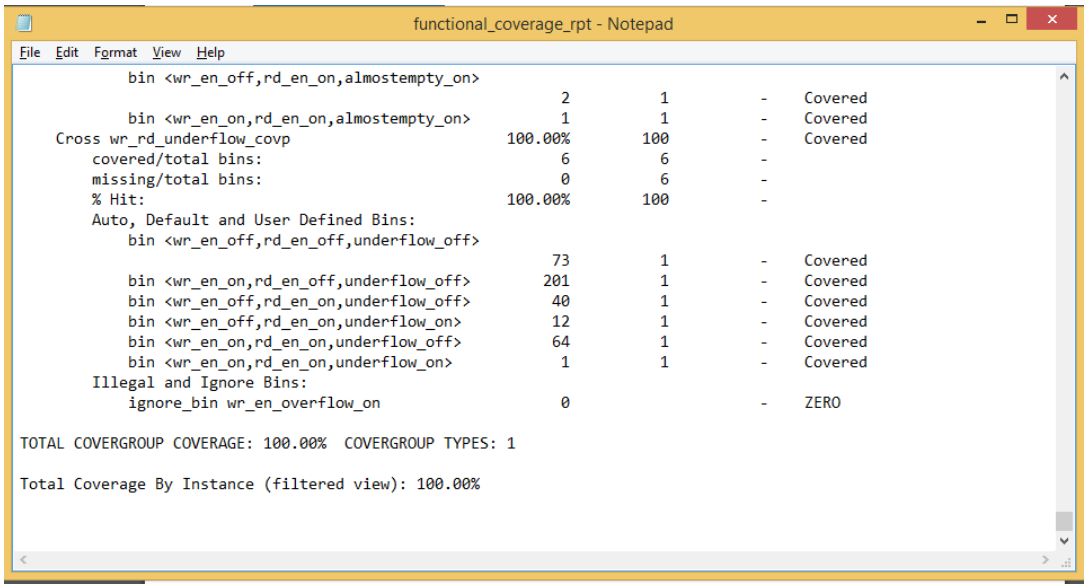


Figure 5 — Functional coverage report: total covergroup coverage 100%.

8. Assertions (Sequential-Domain Coverage)

Fourteen SVA properties are bound to the DUT. The flag checks (full, empty, almostfull, almostempty) are compared against the internal counter in an always_comb block using final deferred assertions, because those flags are combinational. The remaining checks are clocked properties disabled during reset. All assertions passed with zero failures.

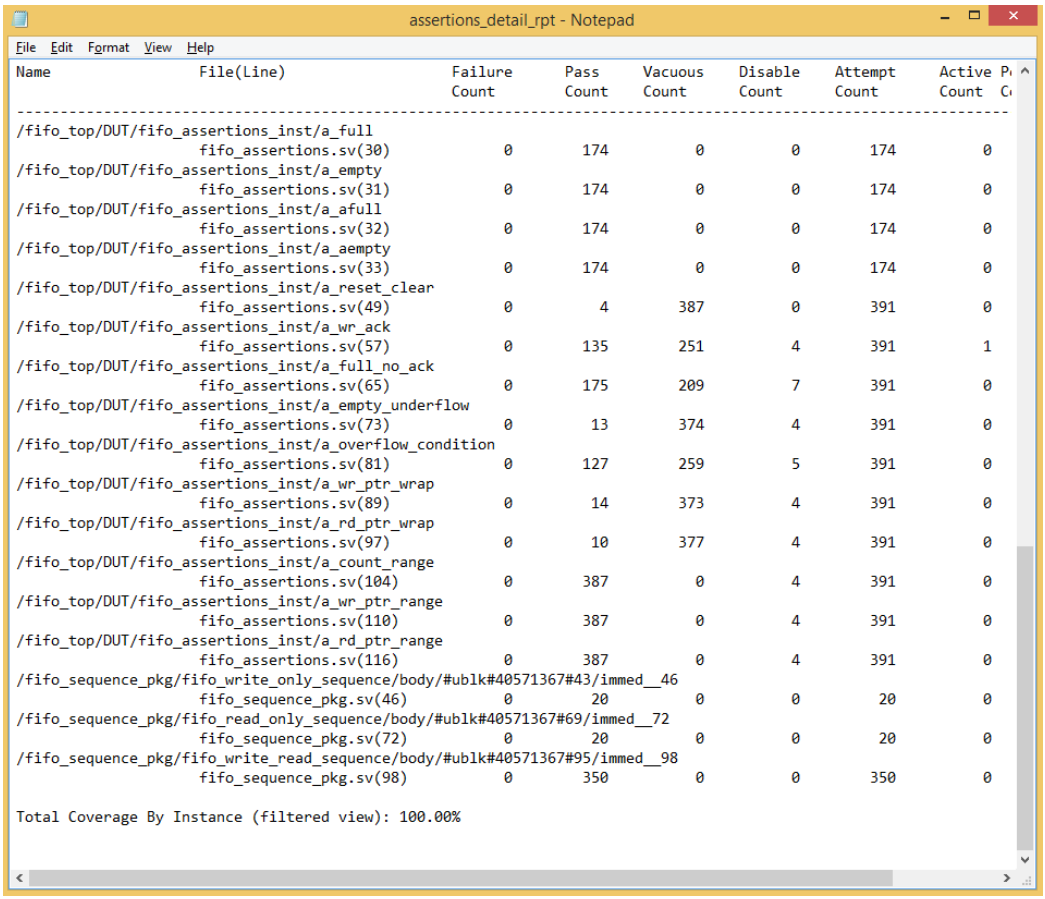


Figure 6 — Assertion detail report: zero failures for all assertions.

Assertions Table

Name	Feature	Assertion	Description
a_reset_clear	**Reset State**	`@(posedge clk) !rst_n |-> (wr_ptr == 0 && rd_ptr == 0 && count == 0 && empty && !full && !wr_ack && !overflow && ! underflow)`	Reset clears pointers, counter, acknowledge and error flags.
a_wr_ack	**Write Ack**	`@(posedge clk) disable iff (!rst_n) (wr_en && !full) |=> wr_ack`	A successful write is acknowledged on the next cycle.
a_full_no_ack	**Full State No Ack**	`@(posedge clk) disable iff (!rst_n) full |=> !wr_ack`	No write acknowledge when the FIFO is full.
a_empty_underflow	**Underflow**	`@(posedge clk) disable iff (!rst_n) (empty && rd_en) |=> underflow`	A read attempt on an empty FIFO raises underflow.
a_overflow_condition	**Overflow**	`@(posedge clk) disable iff (!rst_n) (full && wr_en) |=> overflow`	A write attempt on a full FIFO raises overflow.

Name	Feature	Assertion	Description
a_wr_ptr_wrap	**Write Pointer Wrap**	`@(posedge clk) disable iff (!rst_n) (wr_en && !full && wr_ptr == FIFO_DEPTH-1) |=> (wr_ptr == 0)`	Write pointer wraps to 0 after FIFO_DEPTH-1.
a_rd_ptr_wrap	**Read Pointer Wrap**	`@(posedge clk) disable iff (!rst_n) (rd_en && !empty && rd_ptr == FIFO_DEPTH-1) |=> (rd_ptr == 0)`	Read pointer wraps to 0 after FIFO_DEPTH-1.
a_full	**Full Flag**	`assert final (full == (count == FIFO_DEPTH))`	Full flag matches maximum count (always_comb, checked while rst_n = 1).
a_empty	**Empty Flag**	`assert final (empty == (count == 0))`	Empty flag matches zero count (always_comb).
a_afull	**Almost Full Flag**	`assert final (almostfull == (count == FIFO_DEPTH-1))`	Almost-full flag matches count == FIFO_DEPTH-1 (always_comb).
a_aempty	**Almost Empty Flag**	`assert final (almostempty == (count == 1))`	Almost-empty flag matches count == 1 (always_comb).
a_count_range	**Count Range**	`@(posedge clk) disable iff (!rst_n) (count <= FIFO_DEPTH)`	Counter never exceeds the FIFO capacity.
a_wr_ptr_range	**Write Pointer Range**	`@(posedge clk) disable iff (!rst_n) (wr_ptr < FIFO_DEPTH)`	Write pointer stays inside its legal address range.
a_rd_ptr_range	**Read Pointer Range**	`@(posedge clk) disable iff (!rst_n) (rd_ptr < FIFO_DEPTH)`	Read pointer stays inside its legal address range.

9. Bug Report

The verification environment found ten bugs in the original RTL, grouped below by the area of the design they affect.

#	Area	Bug
1	**Reset**	wr_ack must be low when reset is asserted.
2	**Reset**	overflow must be low when reset is asserted.
3	**Overflow**	overflow must be low when the FIFO is not full.
4	**Reset**	underflow must be low when reset is asserted.
5	**Underflow**	underflow must be low when the FIFO is not empty.
6	**Registered output**	underflow output must be sequential (registered).
7	**Reset**	data_out must be low when reset is asserted.
8	**Simultaneous read/write**	Uncovered case: wr_en and rd_en both high while the FIFO is full — the read must happen.
9	**Simultaneous read/write**	Uncovered case: wr_en and rd_en both high while the FIFO is empty — the write must happen.
10	**Status flag**	almostfull was high with two free locations, while it must be high with only one free location.

10. QuestaSim Waveforms by Sequence

Each sequence item takes one 10 ns clock cycle, so the sequences occupy distinct windows of the simulation: reset 0–10 ns, write-only 10–210 ns, read-only 210–410 ns, and write/read 410–3910 ns. Each snippet below is taken inside its own sequence window.

10.1 Reset Sequence (reset_sequence)

rst_n is driven low at the start of simulation; the pointers and the counter are at 0 and the FIFO reports empty.

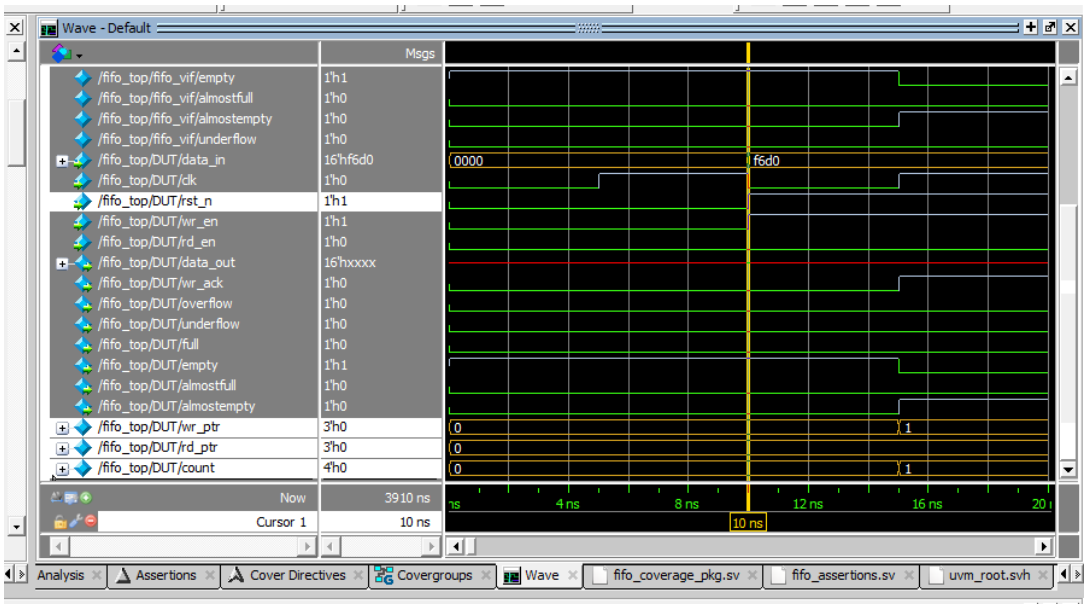


Figure 7 — Reset sequence (0–10 ns).

10.2 Write-Only Sequence (write_sequence)

wr_en is high and rd_en is low. The first write is acknowledged (wr_ack = 1), empty goes low, and almostempty goes high because one word is stored.

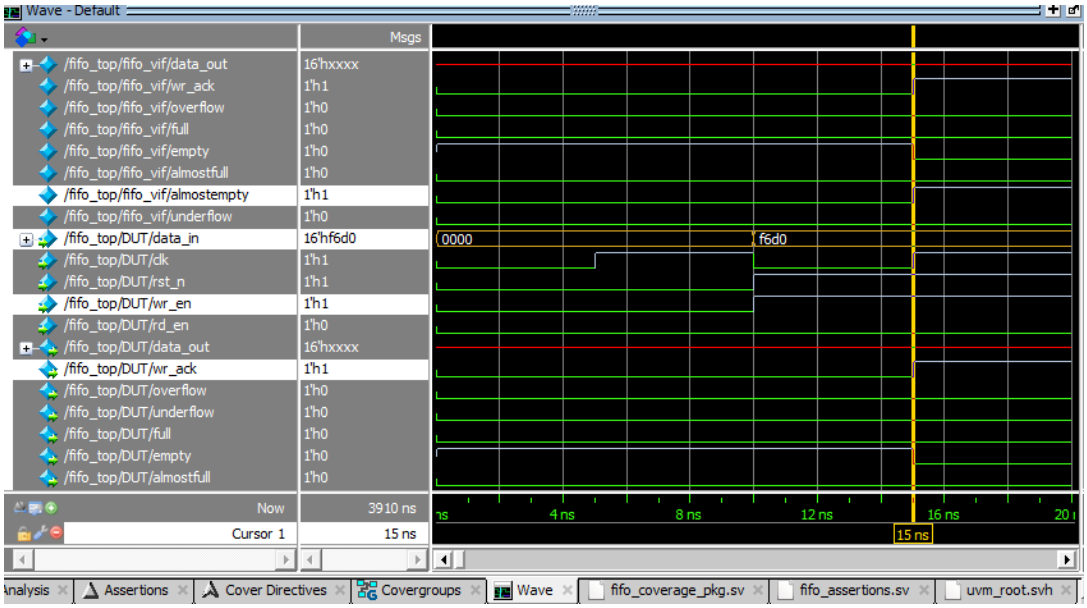


Figure 8 — Write-only sequence (cursor at 15 ns).

10.3 Read-Only Sequence (read_sequence)

rd_en is high and wr_en is low. The first read returns f6d0, the first word written by the write-only sequence, confirming first-in first-out order.

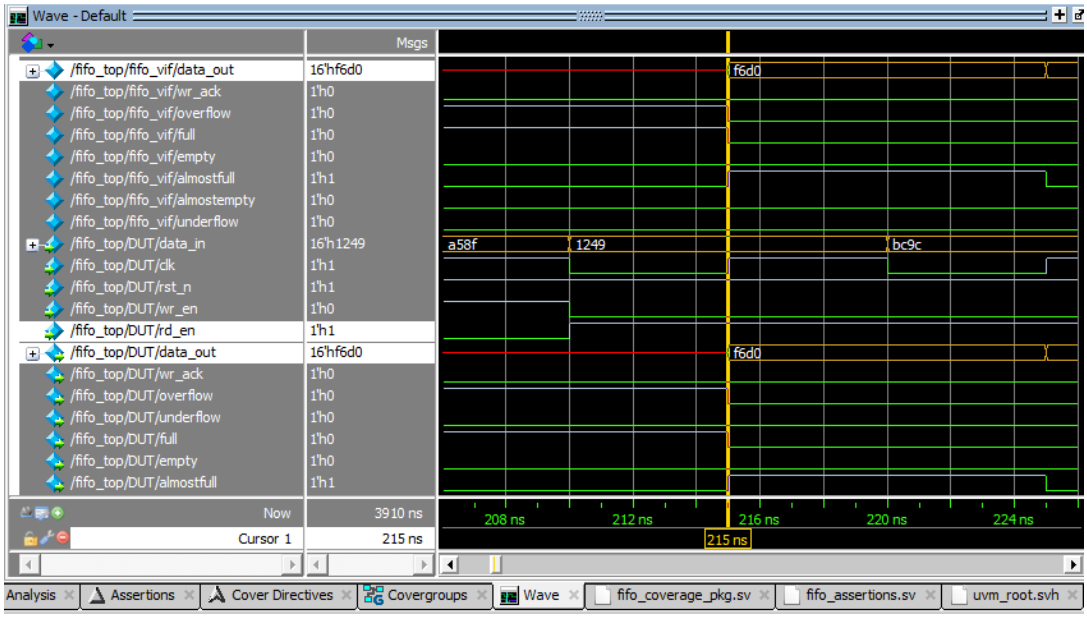


Figure 9 — Read-only sequence (cursor at 215 ns).

10.4 Write/Read Simultaneous Sequence (write_read_sequence)

wr_en and rd_en are randomized together. The pointers wrap around and the counter follows the writes and reads; at the cursor both enables are high while the FIFO is empty (count = 0).

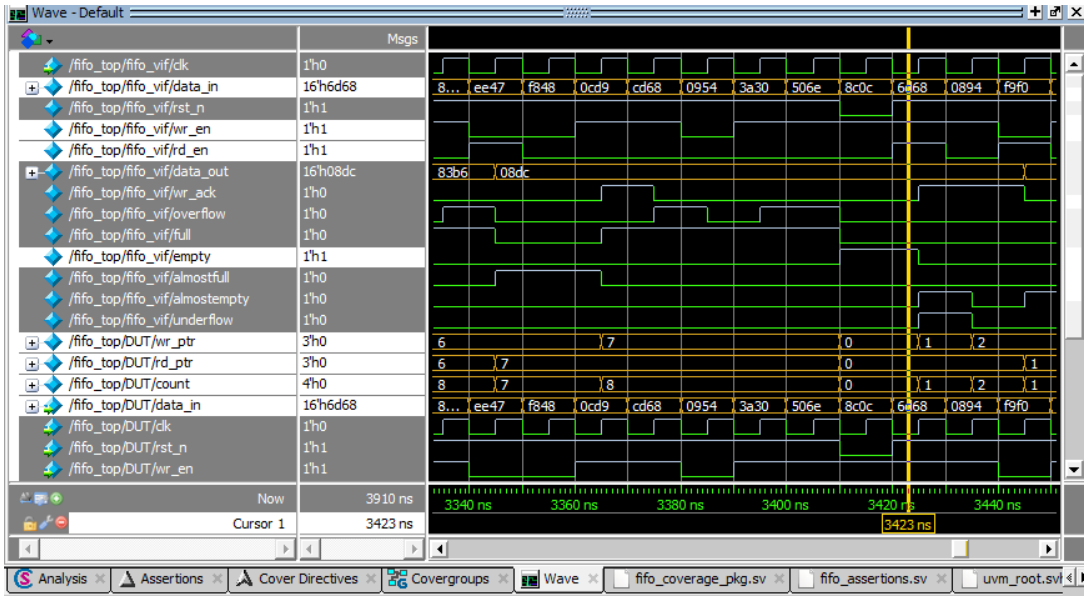


Figure 10 — Write/read sequence (cursor at 3423 ns).

During the same random sequence, a write while the FIFO is full raises full and overflow and the write is not acknowledged.

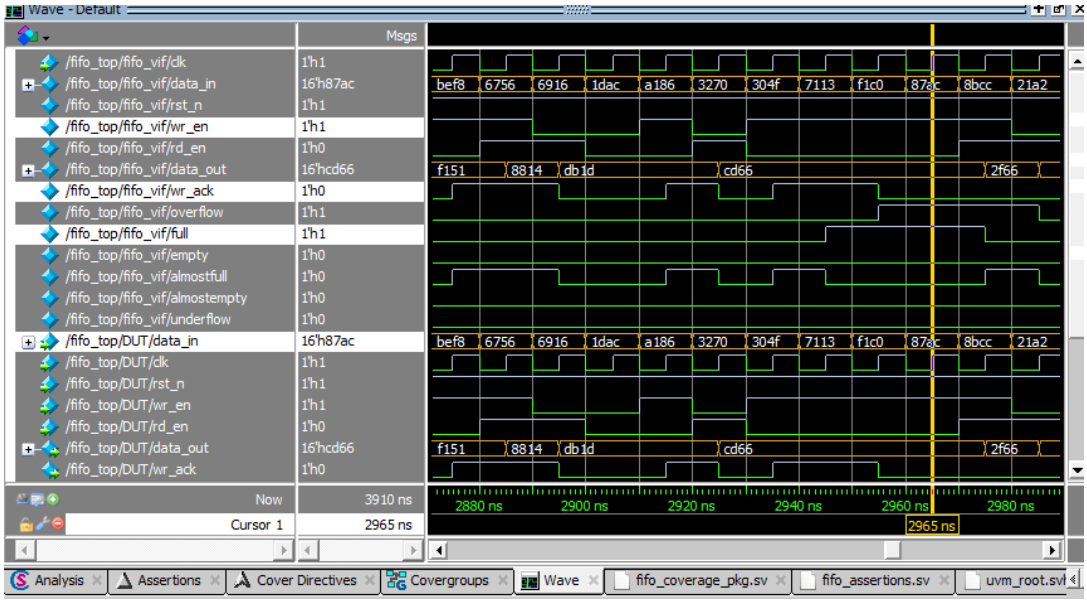


Figure 11 — Write/read sequence: write attempted while full (cursor at 2965 ns).

11. Summary

- Complete UVM environment for a synchronous FIFO: agent, golden-model scoreboard, coverage collector, and bound assertions.
- Four sequences derived from the verification plan: reset, write-only, read-only, and simultaneous write/read.
- 100% functional coverage, 14 SVA assertions passing, and a scoreboard run with zero failed checks.
- Ten bugs found in the original RTL by the environment.